

SmartGate IoT

Práctica 3

Sistema IoT Full-Stack bidireccional: Random Forest, persistencia multivariable y dashboard

Ingeniería en Desarrollo de Software
Internet de las Cosas | Grupo 7P

Lizeth Alexandra Esquivies Torres | 23310409
Caleb Berumen Estrada | 23310408
José Florentino Chavira Sánchez

Objetivo y avance

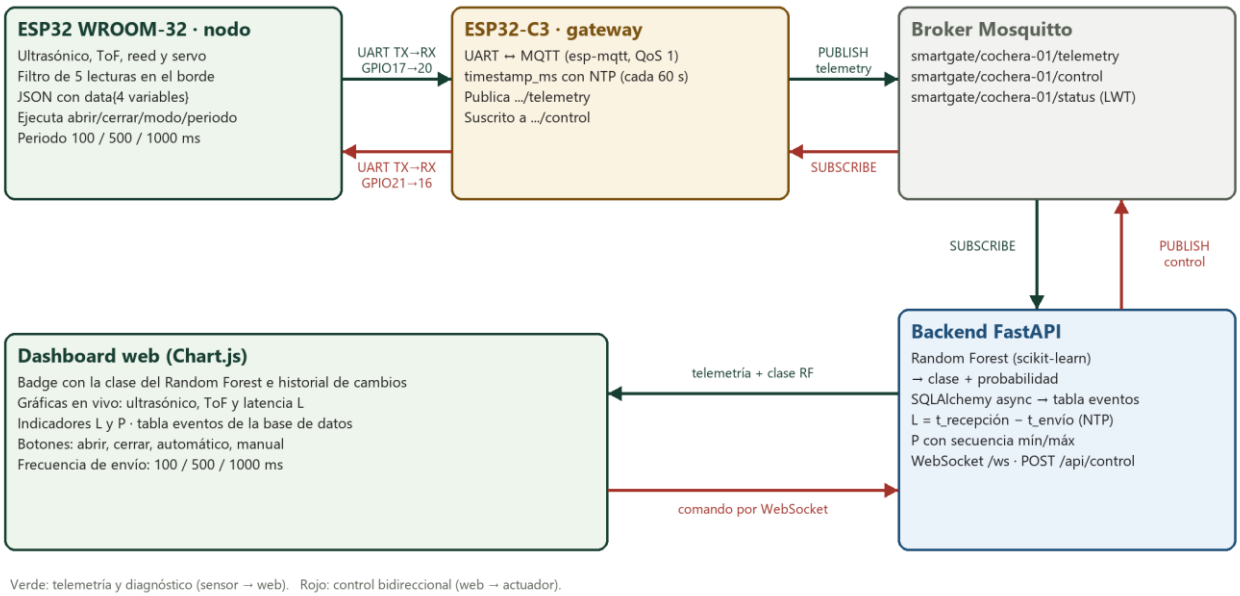
En esta práctica cerramos el ciclo completo de la cochera. La telemetría multivariable sigue viajando de la ESP32 al gateway ESP32-C3, de ahí a Mosquitto y al backend en FastAPI, pero ahora el backend la pasa por un modelo Random Forest que diagnostica el estado del sistema, guarda cada evento con un ORM asíncrono y calcula en vivo la latencia de extremo a extremo L y la pérdida de paquetes P.

El flujo también funciona al revés: desde un dashboard web con Chart.js se puede abrir y cerrar la puerta, cambiar el modo y la frecuencia de envío de la ESP32. El comando viaja por WebSocket, FastAPI lo publica en MQTT, el gateway lo reenvía por UART y la ESP32 lo ejecuta y lo confirma en su siguiente mensaje. Todo se probó con la maqueta real.

Requisitos de la práctica

Requisito	Cómo se cumplió	Estado
Flujo sensor a web con clasificación	Payload con node_id, sequence_id, timestamp_ms y data{4 variables}. FastAPI clasifica cada mensaje con Random Forest y lo reenvía por WebSocket.	Cumplido
Control bidireccional	Botones del dashboard → WebSocket → FastAPI → MQTT smartgate/cochera-01/control → gateway → UART → ESP32.	Cumplido
Persistencia multivariable	SQLAlchemy 2 asíncrono sobre SQLite, tabla eventos con los campos pedidos.	Cumplido
Métricas L y P	L con reloj NTP corregido y P con la fórmula de la guía, en vivo en el dashboard.	Cumplido
Dashboard con Chart.js	Gráficas en vivo, badge del Random Forest, botones de control, L, P y tabla de la base de datos.	Cumplido
Evaluación de red	Corridas de 60 s a 100, 500 y 1000 ms, con la frecuencia cambiada desde el propio canal de control.	Cumplido
Código fuente	Firmware, backend con el modelo y frontend en el repositorio del equipo.	Cumplido
Cita académica	Comparación con Smelcerz (Sensors, 2026).	Cumplido

Arquitectura del sistema



Arquitectura de la práctica 3. En verde, la telemetría con el diagnóstico; en rojo, el control desde la web hasta el actuador.

Respecto a la práctica 2 se agregó un segundo cable entre las placas: el TX del gateway (GPIO21) va al RX2 de la ESP32 (GPIO16). Por ahí bajan los comandos. La ESP32 sigue sin encender su radio, así que el gateway es el único punto de contacto con la red en ambos sentidos.

Payload multivariable

La guía pide node_id, sequence_id, timestamp_ms y un objeto data con las variables. La ESP32 arma data con las cuatro entradas del modelo y el gateway pone timestamp_ms con su hora NTP justo antes de publicar, que es el t_envío de la latencia. El JSON completo conserva además los bloques de las prácticas anteriores (lecturas crudas y filtradas, puerta, vehículo y red).

Variable	Significado
ultrasonico_mm	Distancia en la entrada, promedio de 5 lecturas. 4000 = sin objeto; null = lectura inválida.
tof_mm	Distancia en el estacionamiento. 2000 = sin objeto; null = lectura inválida.
reed_cerrado	1 si el imán está junto al reed.
apertura_pct	Posición ordenada del servo: 0 = cerrado, 100 = abierto.
control	periodo_ms actual y ultimo_cmd, el id del último comando que ejecutó la ESP32.

```
{
  "node_id": "cochera-01",
  "sequence_id": 2126,
  "timestamp_ms": 1792995589306,
  "data": {
    "ultrasonico_mm": 1459.6,
    "tof_mm": 124.2,
    "reed_cerrado": 1,
    "apertura_pct": 0
  },
  "control": {
    "periodo_ms": 500,
    "ultimo_cmd": 2995300
  }
}
```

Campos de la guía tomados de un mensaje real recibido por FastAPI.

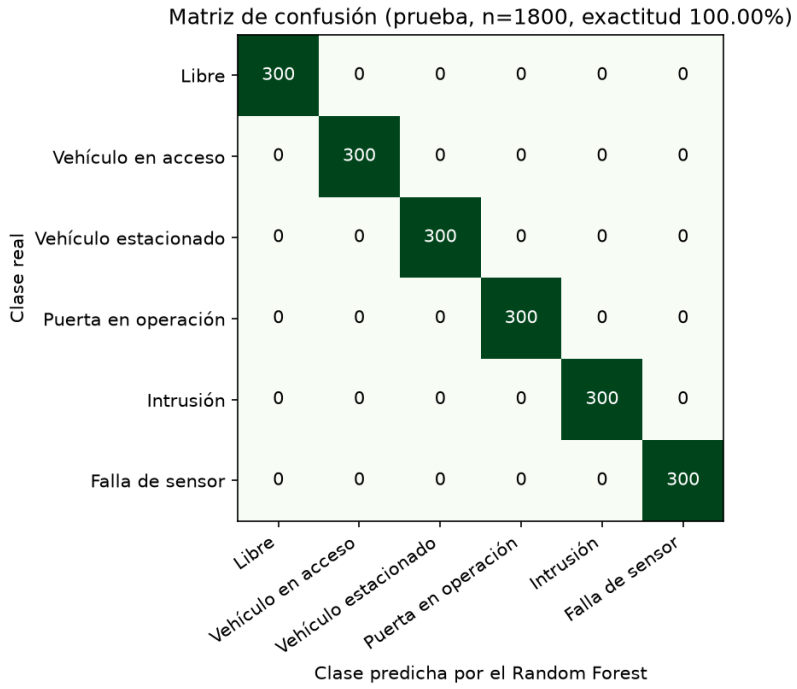
Modelo Random Forest

El modelo diagnostica una de seis condiciones a partir del vector [ultrasonico_mm, tof_mm, reed_cerrado, apertura_pct]. Se entrenó con scikit-learn (100 árboles) y se carga con joblib al arrancar FastAPI.

Clase	Situación que representa
Libre	Puerta cerrada, entrada libre y estacionamiento vacío.
Vehículo en acceso	Algo a menos de 300 mm del ultrasónico: un carro esperando entrar.
Vehículo estacionado	El ToF ve algo dentro (menos de 600 mm) con la puerta cerrada.
Puerta en operación	El servo está moviendo o sosteniendo la puerta abierta.
Intrusión	El reed se abre sin que el servo lo ordene: la puerta fue forzada.
Falla de sensor	El ultrasónico o el ToF entregan una lectura inválida.

La cochera pasa casi todo el tiempo en reposo: de 22 898 lecturas reales, 21 887 son de la clase Libre. Para que el modelo conozca las otras situaciones se tomaron hasta 1 500 lecturas reales por clase y se completó cada clase hasta 1 500 con muestras generadas dentro de los rangos físicos de los sensores. El conjunto final tuvo 2 511 lecturas reales y 6 489 sintéticas, separadas 80/20 para entrenamiento y prueba.

La exactitud en el conjunto de prueba fue 100%, también solo sobre las 501 lecturas reales de prueba. El resultado es tan alto porque las etiquetas de referencia salen de reglas de operación bien definidas; lo que aporta el modelo es aprender esas fronteras a partir de los datos y dar una probabilidad, que sirve para saber qué tan seguro está en los casos límite.



Matriz de confusión del modelo final sobre el conjunto de prueba.

Variable	Importancia en el modelo
tof_mm	32.6%
ultrasonico_mm	29.7%
apertura_pct	19.9%
reed_cerrado	17.7%

Ajuste con una sesión real. La primera versión del modelo se entrenó antes de probar las situaciones físicamente. Después hicimos una sesión con la maqueta: carro en la entrada, carro estacionado, puerta levantada a mano y el cable ECHO del ultrasónico desconectado. Con el carro estacionado, el ToF no midió los 70–100 mm de la semana 1 sino entre 120 y 330 mm, porque el haz solo alcanza una parte del carro. Como el interior vacío siempre da sin objeto, ajustamos la regla a menos de 600 mm y reentrenamos con estos datos.

Clase en la sesión real	Lecturas	Confianza modelo v1	Confianza modelo v2
Libre	673	94%	97%
Vehículo en acceso	5	100%	81%
Vehículo estacionado	246	41%	100%
Puerta en operación	254	96%	100%

Clase en la sesión real	Lecturas	Confianza modelo v1	Confianza modelo v2
Intrusión	18	100%	100%
Falla de sensor	296	98%	100%

El modelo v1, que no conocía la sesión, ya acertaba la clase en las 1 492 lecturas, pero con el carro estacionado lo hacía con apenas 41% de confianza. El v2, que ya incluye la sesión en su entrenamiento, la clasifica con 100%.

Persistencia con ORM asíncrono

La tabla eventos se define con SQLAlchemy 2 y se escribe con una sesión asíncrona sobre aiosqlite, así guardar un registro no bloquea el Event Loop de FastAPI. Cada telemetría produce un registro con id, sequence_id, variables_json, clase_predicha_rf, latencia_ms y fecha_registro, además de la probabilidad, el estado de la puerta y el nodo.

```
sqlite> SELECT id, sequence_id, variables_json, clase_predicha_rf, latencia_ms, fecha_registro
        FROM eventos ORDER BY id DESC LIMIT 6;
id    | seq  | variables_json                                                  | clase_predicha_rf    | latencia | fecha (UTC)
10086 | 2126 | {ultrasonico_mm:1459.6,tof_mm:124.2,reed_cerrado:1,apertura_pct:0.0}   | Vehículo estacionado | 47.6     | 2026-10-26 06:19:50
10085 | 2125 | {ultrasonico_mm:1462.2,tof_mm:125.6,reed_cerrado:1,apertura_pct:0.0}   | Vehículo estacionado | 43.6     | 2026-10-26 06:19:49
10084 | 2124 | {ultrasonico_mm:1461.2,tof_mm:131.2,reed_cerrado:1,apertura_pct:0.0}   | Vehículo estacionado | 44.6     | 2026-10-26 06:19:49
10083 | 2123 | {ultrasonico_mm:1465.8,tof_mm:112.8,reed_cerrado:1,apertura_pct:0.0}   | Vehículo estacionado | 43.6     | 2026-10-26 06:19:48
10082 | 2122 | {ultrasonico_mm:1459.8,tof_mm:114.8,reed_cerrado:1,apertura_pct:0.0}   | Vehículo estacionado | 43.6     | 2026-10-26 06:19:48
10081 | 2121 | {ultrasonico_mm:1461.2,tof_mm:131.0,reed_cerrado:1,apertura_pct:0.0}   | Vehículo estacionado | 46.6     | 2026-10-26 06:19:47
```

Consulta a la tabla eventos. Al cierre de las pruebas había 10 086 registros.

Control bidireccional

Cada botón del dashboard manda por WebSocket un mensaje como {tipo: comando, accion: abrir}. FastAPI lo valida (acciones permitidas y, para la frecuencia, solo 100, 500 o 1000 ms), le asigna un id y lo publica con QoS 1 en smartgate/cochera-01/control. El gateway está suscrito a ese topic y lo reenvía por UART como CMD <id> <acción>. La ESP32 ejecuta la acción y escribe el id en control.ultimo_cmd; cuando ese valor llega en la telemetría, FastAPI marca el comando como confirmado y mide el tiempo de ida y vuelta.

Comando	Lo que hizo la maqueta	Ida y vuelta
manual	Desactivó el ciclo automático.	360 ms
abrir	El servo subió la puerta de 90° a 180°.	354 ms
cerrar	La puerta bajó y el reed confirmó el cierre.	455 ms
auto	Regresó al ciclo automático.	517 ms

El tiempo de ida y vuelta incluye esperar el siguiente mensaje de telemetría, que salía cada 500 ms. El tramo gateway → ESP32 tarda unos 5 ms, como se ve en las consolas.

```
00:15:11.398 CMD -> ESP32: CMD 2995297 manual
00:15:14.440 CMD -> ESP32: CMD 2995298 abrir
00:15:20.508 CMD -> ESP32: CMD 2995299 cerrar
00:15:28.411 CMD -> ESP32: CMD 2995300 auto
```

Consola del gateway: cada comando que llega por MQTT se reenvía a la ESP32.

```
00:15:11.403 CMD #2995297 remoto: manual
00:15:11.403 Modo MANUAL: solo abrir/cerrar por comando.
00:15:14.441 CMD #2995298 remoto: abrir
00:15:14.441 >> ABRIENDO: comando manual
00:15:18.056 ABIERTA.
00:15:20.509 CMD #2995299 remoto: cerrar
00:15:20.509 >> CERRANDO: comando manual
00:15:24.111 CERRADA: el reed confirma el cierre.
00:15:28.515 CMD #2995300 remoto: auto
00:15:28.515 Modo AUTO.
```

Consola de la ESP32: recibe el comando, lo ejecuta y la puerta completa el recorrido.

Métricas de red

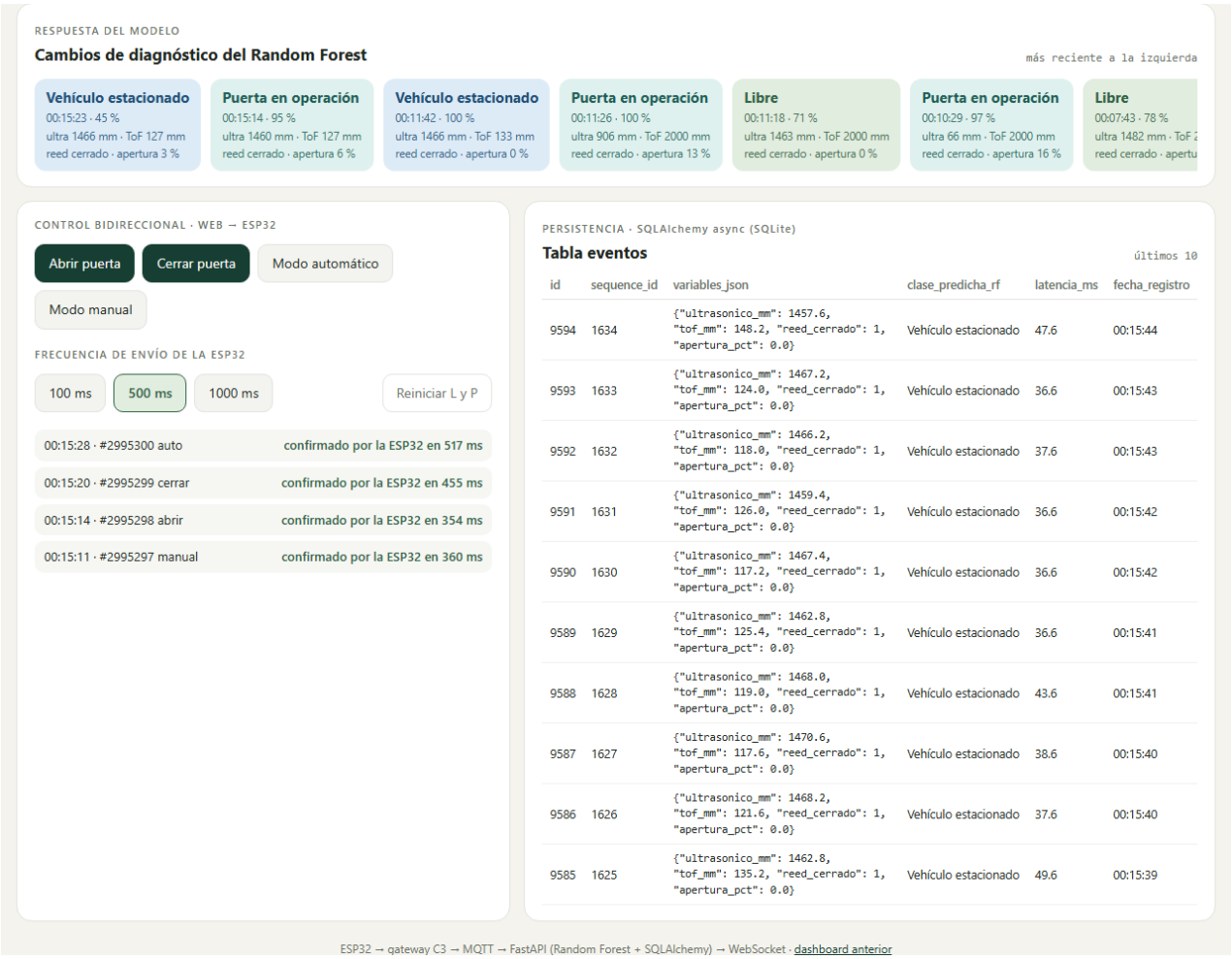
La latencia se calcula como L = t_recepción − t_envío. El t_envío es la hora NTP que pone el gateway y el t_recepción es la hora de la laptop al llegar el mensaje. Como el reloj de la laptop iba alrededor de un segundo adelantado, el backend consulta un servidor NTP al arrancar y cada 10 minutos y corrige t_recepción con ese desfase. Además, el gateway resincroniza su reloj cada minuto, porque al principio se desviaba tanto que daba latencias negativas.

La pérdida usa la fórmula de la guía: P = (1 − recibidos / (secuencia máxima − secuencia mínima + 1)) × 100. Ambas se muestran en vivo y se reinician con un botón para separar corridas.

Dashboard



Dashboard en vivo: diagnóstico del Random Forest, estado de la puerta, L, P y gráficas de los sensores y de la latencia.



Historial de cambios de diagnóstico, comandos confirmados por la ESP32 y tabla eventos de la base de datos.

En el historial se ve la respuesta del modelo ante los cambios de la maqueta: Libre, Puerta en operación al abrir, Vehículo estacionado con el carro dentro. La bitácora muestra los cuatro comandos enviados desde la web y confirmados por la ESP32.

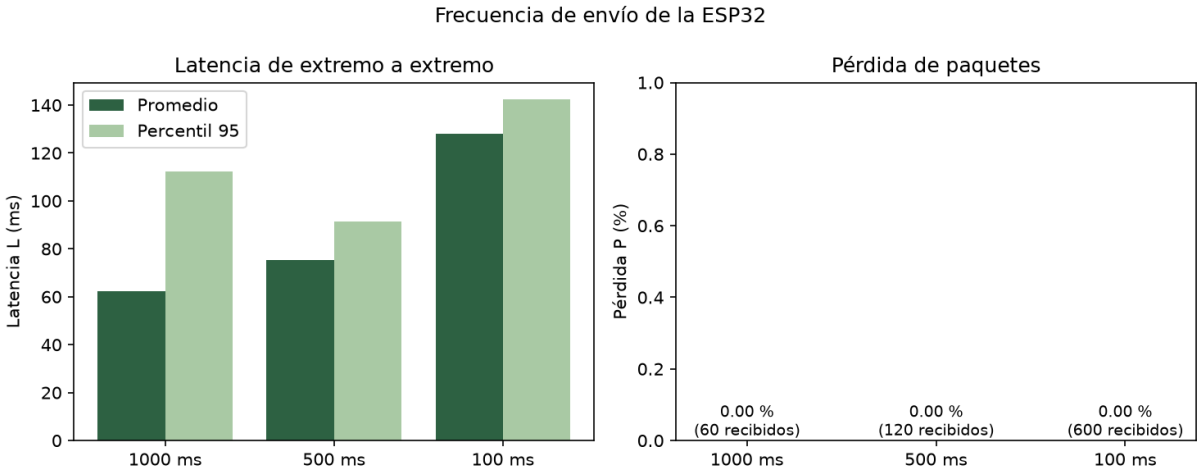
```
00:15:13,613 Guardado cochera-01 #1573 | RF=Vehículo estacionado (1.0, 4832 us) | L=149.6 ms
00:15:14,120 Guardado cochera-01 #1574 | RF=Vehículo estacionado (1.0, 5537 us) | L=154.6 ms
00:15:14,327 Comando #2995298 publicado en smartgate/cochera-01/control: abrir
00:15:14,681 Comando #2995298 abrir confirmado por la ESP32 en 354 ms
00:15:14,681 Guardado cochera-01 #1575 | RF=Puerta en operación (0.95, 5061 us) | L=148.6 ms
00:15:15,203 Guardado cochera-01 #1576 | RF=Puerta en operación (1.0, 5083 us) | L=149.6 ms
00:15:15,722 Guardado cochera-01 #1577 | RF=Puerta en operación (1.0, 5084 us) | L=148.6 ms
00:15:16,244 Guardado cochera-01 #1578 | RF=Puerta en operación (1.0, 4262 us) | L=148.6 ms
00:15:16,765 Guardado cochera-01 #1579 | RF=Puerta en operación (1.0, 4458 us) | L=38.6 ms
00:15:17,286 Guardado cochera-01 #1580 | RF=Puerta en operación (1.0, 5146 us) | L=36.6 ms
00:15:17,806 Guardado cochera-01 #1581 | RF=Puerta en operación (1.0, 5851 us) | L=38.6 ms
00:15:18,266 Guardado cochera-01 #1582 | RF=Puerta en operación (1.0, 5080 us) | L=36.6 ms
00:15:18,767 Guardado cochera-01 #1583 | RF=Puerta en operación (0.99, 4407 us) | L=38.6 ms
00:15:19,269 Guardado cochera-01 #1584 | RF=Puerta en operación (0.99, 4883 us) | L=39.6 ms
00:15:19,769 Guardado cochera-01 #1585 | RF=Puerta en operación (0.99, 5033 us) | L=39.6 ms
00:15:20,266 Guardado cochera-01 #1586 | RF=Puerta en operación (0.99, 4488 us) | L=36.6 ms
```

Consola de FastAPI: clasificación de cada mensaje, comando abrir publicado y confirmado, y cambio de clase a Puerta en operación.

Evaluación experimental de red

Para cada frecuencia, el script de experimento manda el comando de periodo por el mismo canal del dashboard, espera la confirmación de la ESP32, reinicia las métricas y mide durante 60 segundos con la puerta cerrada.

Periodo	Recibidos	P	L promedio	L p95	Jitter	Inferencia RF	Procesamiento
1000 ms	60	0.00 %	62 ms	112 ms	16.7 ms	5.6 ms	11.1 ms
500 ms	120	0.00 %	75 ms	91 ms	7.2 ms	5.4 ms	10.3 ms
100 ms	600	0.00 %	128 ms	142 ms	3.6 ms	5.4 ms	10.3 ms



Latencia y pérdida según la frecuencia de envío de la ESP32.

No se perdió ningún paquete en ninguna frecuencia, ni siquiera a 100 ms con 600 mensajes en un minuto. La latencia promedio subió de 62 ms a 1000 ms y 75 ms a 500 ms hasta 128 ms a 100 ms.

La inferencia del Random Forest no explica ese aumento: tardó entre 5.4 y 5.6 ms y todo el procesamiento del backend (inferencia, SQLite, ORM y reparto por WebSocket) se mantuvo en 10.3–11.1 ms en las tres corridas. El aumento viene de la red y de MQTT: a 100 ms el gateway publica 10 mensajes por segundo con QoS 1, cada uno con su confirmación PUBACK, y los mensajes se encolan en el cliente MQTT y en la pila TCP. Además, el canal UART entre las placas queda ocupado alrededor del 65 % del tiempo (unos 750 bytes por mensaje a 115 200 bps), así que un periodo de 50 ms ya lo saturaría.

Un límite de la medición: el valor absoluto de L depende de que los dos relojes estén sincronizados. En la consola de FastAPI se ve un salto de 148 a 38 ms cuando el gateway resincroniza su NTP, así que estimamos una incertidumbre de unos ±60 ms en L. Las comparaciones entre corridas, el tiempo de procesamiento y el tiempo de ida y vuelta de los comandos no tienen este problema porque se miden con un solo reloj.

Articulación y cita académica

Smelcerz (2026) caracteriza una red de sensores ESP32 muy parecida a la nuestra: nodos que mandan telemetría a un gateway ESP32, que la publica por MQTT hacia un broker Mosquitto y de ahí a la nube [1]. Su aporte es separar la latencia por tramos en lugar de reportar un solo número de extremo a extremo. Reporta de nodo a gateway entre 23.17 y 24.84 ms en promedio (p95 de 28 a 38 ms), un procesamiento en el gateway casi constante de 13.31 a 13.46 ms y una entrega del 100 % con un nodo, que baja a 96.75–97.05 % con cinco nodos.

Nuestros resultados son coherentes con los suyos. El procesamiento en nuestro backend, que además ejecuta el Random Forest y escribe dos veces en la base de datos, fue de 10.3 a 11.1 ms, del mismo orden que su gateway, y se mantuvo constante al subir la carga, igual que en su estudio. Nuestra pérdida fue de 0 % con un solo nodo, que coincide con su 100 % de entrega para un nodo. Nuestra L de 62–128 ms es mayor que su tramo nodo-gateway porque mide más camino: WiFi, broker, el procesamiento en Python y QoS 1 con confirmación.

El artículo también coincide en dos dificultades que vivimos. Primero, la sincronización de relojes: el autor tuvo que diseñar un procedimiento de sincronización y reportar su incertidumbre, y nosotros encontramos latencias negativas hasta resincronizar el gateway cada minuto y corregir el reloj de la laptop. Segundo, señala que una llamada bloqueante puede detener el ciclo del gateway; en la práctica 2 medimos publish() bloqueando hasta 4 s con la red degradada, y por eso aquí la ESP32 que controla la puerta nunca espera a la red.

Problemas y soluciones

Problema	Causa	Solución
La latencia salía negativa.	El reloj del gateway se desviaba después de horas encendido.	SNTP cada 60 s en el gateway y desfase NTP de la laptop medido por el backend.
El modelo confundía la intrusión con falla de sensor.	Los datos sintéticos de intrusión nunca tenían apertura exactamente 0.	Generar intrusiones con el servo en 0 %, como ocurre en la realidad.
Carro estacionado con poca confianza.	El ToF real mide 120–330 mm, no menos de 120.	Regla de 600 mm y reentrenamiento con la sesión real.
El primer comando no se confirmó.	El cable GPIO21 → GPIO16 aún no hacía buen contacto.	Revisión del cable; los siguientes comandos se confirmaron en ~0.5 s.
La hora de la tabla salía en UTC.	SQLite no guarda la zona horaria.	El backend marca la fecha como UTC y el dashboard la muestra en hora local.

Repositorio del equipo

URL: https://github.com/DiabolicAnt/cochera_IoT

Carpeta	Contenido
firmware/SmartGate_ESP32_Cochera	Nodo: sensores, data para el modelo, comandos por UART y periodo variable.
firmware/SmartGate_C3_Gateway	Gateway: telemetría con timestamp_ms y reenvío de comandos.
backend	FastAPI, ml_service.py, db.py (ORM), reloj.py, dashboard y pruebas.
backend/ml	Entrenamiento, modelo .joblib, métricas y evaluación de la sesión real.
backend/experimentos	Experimento de frecuencia de envío.
docs/practica3	Este reporte y las evidencias.

Conclusiones

La cochera ya funciona como un sistema IoT completo en ambos sentidos: los sensores llegan al dashboard con un diagnóstico del Random Forest, cada evento queda guardado en la base de datos y la puerta se puede controlar desde la web con confirmación de la ESP32. En las tres frecuencias probadas no se perdió ningún paquete, y la inferencia del modelo agregó solo unos 5 ms por mensaje.

La lección principal fue probar el modelo con la maqueta real: los datos de la sesión cambiaron nuestra idea de cómo se ve un carro estacionado para el sensor, y reentrenar con ellos subió la confianza del diagnóstico en el caso que más importaba.

Referencias

[1] K. Smelcerz, Characterization of Latency Sources in a MicroPython-Based ESP32 Edge–Cloud Sensor Network, Sensors, vol. 26, no. 17, art. 5555, 2026, doi: 10.3390/s26175555.
[2] F. Pedregosa et al., Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
[3] SQLAlchemy, Asynchronous I/O (asyncio), documentación oficial, versión 2.0.
[4] FastAPI, WebSockets, documentación oficial.
[5] Chart.js, documentación oficial, versión 4.4.